

Week 01 Weekly Exercises

Objectives

- to practice the basics of Rust
- to practice using Rust tools
- to practice rust intro and algebraic types

Activities To Be Completed

The following is a list of all the activities available to complete this week...

- **Mini-Grep**
- **Multiverse**
- **Carnival**
- **DALL-E-RS**
- **Picasso**

The following practice activities are optional and are not **marked, or required** to be completed for the week.

None - all exercises this week are marked.

Preparation

Before attempting the weekly exercises you should re-read the relevant lecture slides and their accompanying examples.

Getting Started

Create a new directory for this week's exercises called `lab01`, change to this directory, and fetch the provided code for this week by running these commands:

```
$ mkdir lab01
$ cd lab01
$ 6991 fetch lab 01
```

Or, if you're not working on CSE, you can download the provided code as a [tar file](#).

EXERCISE:

Mini-Grep

You have been given a crate, `mini_grep`, which prints the first command-line argument.

Your task is to modify `mini_grep/src/main.rs`, such that it reads lines on standard input, printing each line back out if and only if the input line contains the command-line argument.

For example,

```
$ cd mini_grep
$ 6991 cargo run -- foo
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/mini_grep foo`
this line contains the text foo so it is printed
this line contains the text foo so it is printed
... but this line does not, so it is not printed

$ 6991 cargo run -- hi
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/mini_grep foo`
the pattern can appear inside a larger word: chicken!
the pattern can appear inside a larger word: chicken!

$ 6991 cargo run -- HI
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/mini_grep foo`
the pattern is case-sensitive: hi
Hi and hI don't work either
only HI will match this pattern
only HI will match this pattern

$ 6991 cargo run -- 🤔
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/mini_grep foo`
oh and; emojis should work too 🤔🤔🤔
oh and; emojis should work too 🤔🤔🤔
and they follow the same rules: 😊 🌐 📦 🚗
```

HINT:

The documentation for the [contains](#) method may be useful.

The documentation for [if](#) expressions may be useful.

The documentation for [loop](#) expressions may be useful.

The documentation for the [args](#) function may be useful.

You can assume that if [read_line](#) leaves your input string empty ([is_empty](#)) then your program has reached end of input and should exit.

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running give:

```
$ 6991 give--crate
```

You must run give before **Week 2 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with give must be entirely your own.

EXERCISE:

Multiverse

Across the broad range of the multiverse, there are an infinite number of universes! These universes however follow a pattern, allowing us to write code in order to output the details of a universe, given its id.

You have been given a crate, multiverse, which does not compile!

Your task is to modify the given structure, get_universe_details function and the noted areas of the main function, inside multiverse/src/main.rs, such that it loops through id's of 1-15, and prints out the universe details of the universe with that ID.

The universe lookup follows some basic rules:

- If the universe id is **divisible by 3** - then that universe is the star wars universe, where the rebels won and the population is the max number that a u32 can fit!
- If the universe id is **divisible by 5** - then it is the miraculous ladybug universe, where the villian Hawk Moth won, leaving the universe with a population of 22.
- Finally, if the universe id is **divisible by both 5 and 3** - then it is the Stardew Valley universe, where the evil Jojo Corp won, leaving a population of only one!

Your program should match the following example

```
$ cd multiverse
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/multiverse`
Universe with id 1 is unknown
Universe with id 2 is unknown
Universe with id 3 is called Star Wars, won by The Rebellion and has a population of 4294967295
Universe with id 4 is unknown
Universe with id 5 is called Miraculous, won by Hawk Moth and has a population of 22
Universe with id 6 is called Star Wars, won by The Rebellion and has a population of 4294967295
Universe with id 7 is unknown
Universe with id 8 is unknown
Universe with id 9 is called Star Wars, won by The Rebellion and has a population of 4294967295
Universe with id 10 is called Miraculous, won by Hawk Moth and has a population of 22
Universe with id 11 is unknown
Universe with id 12 is called Star Wars, won by The Rebellion and has a population of 4294967295
Universe with id 13 is unknown
Universe with id 14 is unknown
Universe with id 15 is called Stardew Valley, won by Jojo Corp and has a population of 1
```

HINT:

Click to reveal further hint

The documentation for the [if](#) keyword may be useful.

The documentation for the [struct](#) keyword may be useful.

The documentation for the [option](#) module may be useful.

NOTE:

Authors Note

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running give:

```
$ 6991 give--crate
```

You must run give before **Week 2 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with give must be entirely your own.

EXERCISE:

Carnival

Shrey and Tom are looking to go to the carnival! However, there is only one ticket 🥺

You have been provided a crate, `carnival`. Your task is to modify the main function in `carnival/src/main.rs`, and the `move_ticket` function, to make them go to the carnival.

Specifically, you should

- Only change the main function from line 16 onwards
- Implement the `move_ticket` function
- Move the ticket that Shrey owns to Tom
- Not use any concepts not yet taught (namely, borrowing)

This exercise aims to test your understanding of Rust's ownership system. Like most exercises, you can simply just print the right output, or do lots of `.clone()`, but this will not help you in future assignments or the final exam.

The output when completed, should look like:

```
$ cd carnival
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/carnival`
SHREY ticket: None
TOM ticket: Some(Ticket)
```

HINT:

Hint 1

Hint 2

When you think your program is working, you can use `autotest` to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running `give`:

```
$ 6991 give-crate
```

You must run `give` before **Week 2 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with `give` must be entirely your own.

EXERCISE:

DALL-E-RS

In order to get started, you will need to create a Cargo project

You can do this by running the `cargo new` command in your terminal

```
$ 6991 cargo new dall-e-rs
Created binary (application) `dall-e-rs` package
```

NOTE:

Despite other activities in week 1 using cargo, this exercise description goes into some more depth about how cargo works.

Your task is to create a simple **Cargo project**, in order to generate some art by outputting a 200 x 200 bmp image. This exercise enables you to practice using the cargo build system. It is manually marked, (practically any image that has some degree of effort will get full marks) and **your submission will be uploaded to a course gallery** later during week 2/3.

You should now have a Cargo package inside the `dall-e-rs` directory.

```
$ cd dall-e-rs
$ ls
Cargo.toml  src
```

The cargo project starts you off with a `Cargo.toml` file, which describes the project.

```
$ cat Cargo.toml
[package]
name = "dall-e-rs"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
```

The `dependencies` section of the `Cargo.toml` file describes the dependencies of the project.

In this case we need to have a dependency on the `bmp` crate (remember, a crate is what Rust calls an external library).

We can add this dependency by running the following command:

```
$ 6991 cargo add bmp
Updating crates.io index
Adding bmp v0.5.0 to dependencies.
```

You can search for Rust libraries (we call them "crates") on the Rust community crate registry: <https://crates.io/>. To view the documentation for a particular crate, you can visit `https://docs.rs/<crate>/`, where `<crate>` is the name of the crate.

For example, the documentation for the `bmp` crate can be found at <https://docs.rs/bmp/>.

Cargo also creates a `src` directory, which contains the source code for the project. This directory starts with a `main.rs` file, which is the entry point for the project.

```
$ ls src
main.rs
$ cat src/main.rs
fn main() {
    println!("Hello, world!");
}
```

You write your program in here!

To compile your project, you can run the following command:

```
$ 6991 cargo build
    Updating crates.io index
   Compiling byteorder v1.4.3
   Compiling bmp v0.5.0
   Compiling dall-e-rs v0.1.0 (/tmp/tmp.CCQoWnQuRt/dall-e-rs)
    Finished dev [unoptimized + debuginfo] target(s) in 1.46s
```

Your code will be compiled into an executable binary, located in the newly created `target` directory.

```
$ ls
Cargo.lock  Cargo.toml  src  target
$ ls target
CACHEDIR.TAG  debug
$ ls target/debug
build  deps  examples  incremental  dall-e-rs  dall-e-rs.d
$ ./target/debug/dall-e-rs
Hello, world!
```

Instead of writing `6991 cargo build`, followed by `./target/debug/dall-e-rs` each time you want to run your program, you can simply use the `6991 cargo run` command.

```
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/dall-e-rs`
Hello, world!
```

Now it is your turn! Think of a fun idea for a simple piece of art, and then write it in the `src/main.rs` file. Your code should generate a single file, which should have a `.bmp` extension.

Don't forget to test your code along the way with the `cargo run` command.

When you are finished working on this exercise, you must submit your work by running `give`:

```
$ 6991 give-crate
```

You must run `give` before **Week 2 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with `give` must be entirely your own.

EXERCISE: Picasso

Your task is to write a small command-line utility, that will read in a list of file names (`.bmp` files) and output their representation to the terminal.

Your program will only be given either valid BMP files, consisting of RED, LIME, BLUE and WHITE pixels, or a malformed file (including, for example, a filepath to a file that does not exist). These constants can be found in [bmp::consts](#) (i.e, the `consts` module of the `bmp` crate).

If your program is given a valid BMP file, it should output the following:

```
$ 6991 cargo run -- path/to/shreys_art.bmp
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/picasso path/to/shreys_art.bmp`
===== ../../files.cp/shreys_art.bmp =====
R W W W W W W W W W W W W B G W R W W R W W R W W R W W R
W R W W W W W W W W W W W B G R W W R W W R W W R W W R W
W W R W W W W W W W W W W B G W W R W W R W W R W W R W W
W W W R W W W W W W W W W B G W R W W R W W R W W R W W R
W W W W R W W W W W W W W B G R W W R W W R W W R W W R W
W W W W W R W W W W W W W B G W W R W W R W W R W W R W W
W W W W W W R W W W W W W B G W R W W R W W R W W R W W R
W W W W W W W R W W W W W B G R W W R W W R W W R W W R W
W W W W W W W W R W W W W B G W W R W W R W W R W W R W W
W W W W W W W W W R W W W B G W R W W R W W R W W R W W R
W W W W W W W W W W R W W B G W W R W W R W W R W W R W W
W W W W W W W W W W W R W B G W R W W R W W R W W R W W R
W W W W W W W W W W W W R B G R W W R W W R W W R W W R W
G G G G G G G G G G G G G B G B B B B B B B B B B B B B
B B B B B B B B B B B B B B G B G G G G G G G G G G G G
W R W W R W W R W W R W W R G B R W W W W W W W W W W W W
R W W R W W R W W R W W R W G B W R W W W W W W W W W W W
W W R W W R W W R W W R W W G B W W R W W W W W W W W W W
W R W W R W W R W W R W W R G B W W W R W W W W W W W W W
R W W R W W R W W R W W R W G B W W W W R W W W W W W W W
W W R W W R W W R W W R W W G B W W W W W R W W W W W W W
W R W W R W W R W W R W W R G B W W W W W W W W W W W W W
W W R W W R W W R W W R W W G B W W W W W W W W W W R W W
W R W W R W W R W W R W W R G B W W W W W W W W W W W R W
R W W R W W R W W R W W R W G B W W W W W W W W W W W W R
```

If your program is given a malformed BMP file, it should output Error!, followed by the debug representation of the error message. e.g. - consider the following error variation, where an invalid compression type is given:

```
$ 6991 cargo run -- path/to/bad_compression.bmp
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/picasso ../../files.cp/bad_compression.bmp`
===== path/to/bad_compression.bmp =====
Error! BmpError { kind: UnsupportedCompressionType, details: "RLE 8-bit" }
```

Multiple files should be seperated by equals and the filename, e.g.:


```
$ 6991 cargo run -- path/to/bad_compression.bmp path/to/shreys_art.bmp
    Finished dev [unoptimized + debuginfo] target(s) in 0.44s
    Running `target/debug/picasso path/to/bad_compression.bmp path/to/shreys_art.bmp`
===== path/to/bad_compression.bmp =====
Error! BmpError { kind: UnsupportedCompressionType, details: "RLE 8-bit" }
===== path/to/shreys_art.bmp =====
R W W W W W W W W W W W W W B G W R W W R W W R W W R
W R W W W W W W W W W W W W B G R W W R W W R W W R W
W W R W W W W W W W W W W W B G W W R W W R W W R W W
W W W R W W W W W W W W W W B G W R W W R W W R W W R
W W W W R W W W W W W W W W B G R W W R W W R W W R W
W W W W W R W W W W W W W W B G W W R W W R W W R W W
W W W W W W R W W W W W W W B G W R W W R W W R W W R
W W W W W W W R W W W W W W B G R W W R W W R W W R W
W W W W W W W W R W W W W W B G R W W R W W R W W R W
W W W W W W W W W R W W W W B G W W R W W R W W R W W
W W W W W W W W W W R W W B G W W R W W R W W R W W W
W W W W W W W W W W W R W B G W R W W R W W R W W R W
W W W W W W W W W W W W R B G R W W R W W R W W R W W
G G G G G G G G G G G G G B G B B B B B B B B B B B
B B B B B B B B B B B B B G B G G G G G G G G G G G
W R W W R W W R W W R W W R G B R W W W W W W W W W W
R W W R W W R W W R W W R W G B W R W W W W W W W W W
W W R W W R W W R W W R W W G B W W R W W W W W W W W
W R W W R W W R W W R W W R G B W W W R W W W W W W W
R W W R W W R W W R W W R W G B W W W W W W W W W W W
R W W R W W R W W R W W R W G B W W W W W W W W W W R
W W R W W R W W R W W R W W G B W W W W W W W W W W R
W R W W R W W R W W R W W R G B W W W W W W W W W W R
R W W R W W R W W R W W R W G B W W W W W W W W W W R
```

HINT:

This is a pretty beefy exercise, break it down into steps! Make sure to look at the below documentation, especially the examples!

Note: LIME should display as G

You should aim to do this exercise without borrowing, or other advanced concepts that have not yet been taught. If you are unsure, ask on the course forum or in consultations.

The documentation for the [match](#) keyword may be useful.

The documentation for the [result](#) type may be useful.

The documentation for the [bmp](#) crate may be useful.

The documentation for the [get_pixel](#) method may be useful.

The documentation for the [args](#) function may be useful.

Click to reveal further hint

When you think your program is working, you can use autotest to run some simple automated tests:

```
$ 6991 autotest
```

When you are finished working on this exercise, you must submit your work by running give:

```
$ 6991 give--crate
```

You must run give before **Week 2 Wednesday 21:00:00** to obtain the marks for this exercise. Note that this is an individual exercise; the work you submit with give must be entirely your own.

Submission

When you are finished each exercise make sure you submit your work by running give.

You can run give multiple times. Only your last submission will be marked.

Don't submit any exercises you haven't attempted.

If you are working at home, you may find it more convenient to upload your work via [give's web interface](#).

Remember you have until **Week 2 Wednesday 21:00:00** to submit your work.

You cannot obtain marks by e-mailing your code to tutors or lecturers.

Automarking will be run several days after the submission deadline, using test cases different to those autotest runs for you.
(Hint: do your own testing as well as running autotest.)

After automarking is run you can [view your results here](#).

Weekly exercise Marks

When all exercises of a week are automarked you should be able to view the the marks [via give's web interface](#) or by running this command on a CSE machine:

```
$ 6991 classrun -sturec
```

COMP6991 23T1: Solving Modern Programming Problems with Rust is brought to you by
the [School of Computer Science and Engineering](#)
at the [University of New South Wales](#), Sydney.

For all enquiries, please email the class account at cs6991@cse.unsw.edu.au

CRICOS Provider 00098G

[Login as tutor](#)